Practical No. 1

Definition : Write a program to demonstrate basic input and output operations in Python.

Aim/ Objective:

- To understand how to accept input from the user in Python.
- To display output using the print() function.
- To perform basic operations on user-provided data.
- To understand basic Python syntax for input and output.

Source Code:

```
# Program to demonstrate basic input and output operations

# Taking input from the user
name = input("Enter your name: ")
age = int(input("Enter your age: "))

# Displaying output
print("\n--- User Details ---")
print("Name:", name)
print("Age:", age)

# Taking two numbers as input
num1 = float(input("\nEnter first number: "))
num2 = float(input("Enter second number: "))

# Performing basic arithmetic operation
sum = num1 + num2

# Displaying the result
print("Sum of two numbers:", sum)
```

Output:

Enter your name: vasu

Enter your age: 40

--- User Details ---

Name: vasu

Age: 40

Enter first number: 25

Enter second number: 15

Sum of two numbers: 40.0

Conclusion:

The program successfully demonstrates the basic input and output operations in Python. It shows how to accept data from the user using input(), convert input into appropriate data types, store values in variables, perform a basic arithmetic operation, and display the results using print()

Practical No. 2

Definition : Write a program to illustrate the use of different data types and type casting.

Aim/ Objective:

- To understand different data types in Java.
- To learn how to declare and initialize variables of different data types.
- To understand **type casting** in Java.
- To demonstrate **implicit** and **explicit** type casting.
- To observe how data is converted from one data type to another.

Source Code:

Program to demonstrate basic input and output operations

```
# Output
print("Welcome to Python!")

# Input
name = input("Enter your name: ")
age = input("Enter your age: ")

# Output
print("\n--- User Details ---")
print("Name:", name)
print("Age:", age)

print("Hello,", name + "! Nice to meet you.")
```

Output:

Integer: 10

Float: 20.5

Double: 30.75

Character: A

Boolean: true

Implicit Type Casting:

Integer value: 100

Double value: 100.0

Explicit Type Casting:

Double value: 25.75

Integer value: 25

Conclusion:

The program successfully demonstrates different data types in Java and shows both implicit type casting and explicit type casting.

Practical No. 3

Definition : Write a program to perform arithmeti relational and logical operations using Pythono perators.

Aim/ Objective:

- **To understand arithmetic operators in Python.**
- **To perform relational operations between two values.**
- **To understand and use logical operators.**
- **To learn how Python operators are used in expressions.**

Source Code:

```
# Program to demonstrate Arithmetic, Relational and Logical Operators

a = 10
b = 5

# Arithmetic Operators
print("Arithmetic Operations:")
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Modulus:", a % b)
print("Exponent:", a ** b)
print("Floor Division:", a // b)
```

```
# Relational Operators
print("\nRelational Operations:")
print("a == b:", a == b)
print("a != b:", a != b)
print("a > b:", a > b)
print("a < b:", a < b)
print("a >= b:", a >= b)
print("a <= b:", a <= b)

# Logical Operators
x = True
y = False

print("\nLogical Operations:")
print("x and y:", x and y)
print("x or y:", x or y)
print("not x:", not x)
```

Output:

Arithmetic Operations:

Addition: 15

Subtraction: 5

Multiplication: 50

Division: 2.0

Modulus: 0

Exponent: 100000

Floor Division: 2

Relational Operations:

a == b: False

a != b: True

a > b: True

a < b: False

a >= b: True

a <= b: False

Logical Operations:

x and y: False

x or y: True

not x: False

Conclusion:

The program successfully demonstrates arithmetic, relational, and logical operators in Python and shows how these operators are used to perform calculations, comparisons, and logical operations.

Practical No. 4

Definition : Write a program to demonstrate string operations including slicing formattingand built-in string functions.

Aim/Objective:

- **To understand and perform basic string operations in Python.**
- **To demonstrate string slicing to access specific characters.**
- **To use string formatting techniques.**
- **To apply built-in string functions for manipulating strings.**
- **To understand efficient handling and processing of strings in Python.**

Source Code:

```
# Program to demonstrate String Operations


text = "Python Programming"


print("Original String:", text)
```

Enrolment/ Roll No. 92600584091

```
# String Slicing
print("\n--- String Slicing ---")
print("First 6 characters:", text[:6])
print("Last 11 characters:", text[7:])
print("Characters from index 2 to 8:", text[2:9])
print("Reverse string:", text[::-1])

# String Formatting
print("\n--- String Formatting ---")
name = "Rahul"
age = 20

print("My name is {} and I am {} years old.".format(name, age))
print(f"My name is {name} and I am {age} years old.")

# Built-in String Functions
print("\n--- Built-in String Functions ---")
print("Length:", len(text))
print("Uppercase:", text.upper())
print("Lowercase:", text.lower())
print("Title Case:", text.title())
print("Replace:", text.replace("Python", "Java"))
print("Find:", text.find("Programming"))
print("Count of m:", text.count("m"))
print("Starts with Python:", text.startswith("Python"))
print("Ends with ing:", text.endswith("ing"))
```

Output:

```
Original String: Python Programming

--- String Slicing ---
```

Enrolment/ Roll No. 92600584091

First 6 characters: Python

Last 11 characters: Programming

Characters from index 2 to 8: thon Pr

Reverse string: gnimmargorP nohtyP

--- String Formatting ---

My name is Rahul and I am 20 years old.

My name is Rahul and I am 20 years old.

--- Built-in String Functions ---

Length: 18

Uppercase: PYTHON PROGRAMMING

Lowercase: python programming

Title Case: Python Programming

Replace: Java Programming

Find: 7

Count of m: 2

Starts with Python: True

Ends with ing: True

Conclusion:

Thus, the Python program successfully demonstrates string slicing, string formatting, and various built-in string functions such as `len()`, `upper()`, `lower()`, `replace()`, `find()`, `count()`, `startswith()`, and `endswith()`.

Practical No. 5

Definition: Write a program to create and manipulate lists using indexing slicing and list comprehensions.

Aim/Objectives:

- **To create and manipulate lists in Python.**
- **To access list elements using indexing.**
- **To extract elements using slicing.**
- **To create new lists using list comprehensions.**
- **To understand basic list manipulation techniques efficiently.**

Source Code:

```
numbers=[10,20,30,40,50]
print("Original Numbers:",numbers)
print("Number First:",numbers[1])
print("Number Third:",numbers[2])
print("Last Number:",numbers[-1])
print("Last three elements:",numbers[2:5])
print("First three elements:",numbers[0:3])
print("First element:",numbers[0])
numbers.append(60)
print("After appened:",numbers)
numbers.remove(10)
print("After remove:",numbers)
numbers[0]=25
```

Enrolment/ Roll No. 92600584103

```
print("After changing the value:",numbers)
squares=[x*x for x in numbers]
print("Squares Numbers are:",squares)
even_numbers=[x for x in numbers if x %2==0]
print("Even Numbers are:",even_numbers)
```

Output:

Original Numbers: [10, 20, 30, 40, 50]

Number First: 20

Number Third: 30

Last Number: 50

Last three elements: [30, 40, 50]

First three elements: [10, 20, 30]

First element: 10

After appened: [10, 20, 30, 40, 50, 60]

After remove: [20, 30, 40, 50, 60]

After changing the value: [25, 30, 40, 50, 60]

Squares Numbers are: [625, 900, 1600, 2500, 3600]

Even Numbers are: [30, 40, 50, 60]

Conclusion:

To understand how to access, modify, and extract elements from a list using indexing and slicing, and to create new lists efficiently using list comprehensions.

Practical No. 6

Definition:

Write a program to illustrate the use of tuples and sets with basic operations.

Aim/Objectives:

- **To understand the use of tuples and sets in Python.**
- **To create and initialize tuples and sets.**

Enrolment/ Roll No. 92600584103

- **To perform basic operations on tuples.**
- **To perform basic set operations such as union, intersection, and difference.**
- **To understand the importance of tuples and sets in Python.**

Source Code:

```
# Tuple
numbers = (10, 20, 30, 40, 50)
print("Tuple:", numbers)
print("First element:", numbers[0])
```

```
print("Last element:", numbers[-1])

print("Sliced tuple:", numbers[1:4])

print("Length of tuple:", len(numbers))

print("Count of 20:", numbers.count(20))

print("Index of 30:", numbers.index(30))

set1 = {10, 20, 30, 40}

set2 = {30, 40, 50, 60}

print("\nSet 1:", set1)

print("Set 2:", set2)

set1.add(70)

print("After adding 70:", set1)

set1.remove(20)

print("After removing 20:", set1)

print("Union:", set1.union(set2))

print("Intersection:", set1.intersection(set2))

print("Difference:", set1.difference(set2))
```

Output:

```
Tuple: (10, 20, 30, 40, 50)

First element: 10

Last element: 50

Sliced tuple: (20, 30, 40)
```

Enrolment/ Roll No. 92600584103

Length of tuple: 5

Count of 20: 1

Index of 30: 2

Set 1: {40, 10, 20, 30}

Set 2: {40, 50, 60, 30}

After adding 70: {70, 40, 10, 20, 30}

After removing 20: {70, 40, 10, 30}

Union: {70, 40, 10, 50, 60, 30}

Intersection: {40, 30}

Difference: {10, 70}

Conclusion:

Thus, we learned how to create and manipulate tuples and sets using basic operations.

We also understood their differences and practical uses in Python.

Practical No. 7

Definition:

Write a program to create a dictionary and demonstrate dictionary methods and iteration.

Aim/Objectives:

- **To understand the concept of dictionaries in Python.**
- **To create and initialize a dictionary.**

- **To perform various dictionary methods.**
- **To iterate through dictionary keys and values.**
- **To understand the use of dictionaries for storing key-value pairs.**

Source Code:

```
# Create a dictionary
student = {
"RollNo": 101,

Enrolment/ Roll No. 92600584103
"Name": "jaynish",
"Course": "MCA",
"Marks": 85
}
print("Original Dictionary:", student)
# Accessing values
print("Student Name:", student["Name"])
# Dictionary methods
print("Keys:", student.keys())
print("Values:", student.values())
print("Items:", student.items())
# Add a new item
student["Grade"] = "A"
print("After adding Grade:", student)
```

```
# Update an item
student["Marks"] = 90
print("After updating Marks:", student)
# Remove an item
student.pop("Grade")
print("After removing Grade:", student)
# Iteration through dictionary
print("\nDictionary Elements:")
for key, value in student.items():
print(key, ":", value)
```

Output:

```
Original Dictionary: {'RollNo': 101, 'Name': 'jaynish', 'Course': 'MCA', 'Marks': 85}
Student Name: jaynish
Keys: dict_keys(['RollNo', 'Name', 'Course', 'Marks'])
Values: dict_values([101, 'jaynish', 'MCA', 85])
Items: dict_items([('RollNo', 101), ('Name', 'jaynish'), ('Course', 'MCA'), ('Marks', 85)])
After adding Grade: {'RollNo': 101, 'Name': 'jaynish', 'Course': 'MCA', 'Marks': 85,
'Grade':
'A'}
After updating Marks: {'RollNo': 101, 'Name': 'jaynish', 'Course': 'MCA', 'Marks': 90,
'Grade':
'A'}
After removing Grade: {'RollNo': 101, 'Name': 'jaynish, 'Course': 'MCA', 'Marks': 90}
```

Dictionary Elements:

RollNo : 101

Name : jaynish

Course : MCA

Marks : 90

Conclusion:

Thus, we learned how to create and manipulate dictionaries using different methods and

iteration. We also understood how to access and process key-value pairs efficiently in Python.

Practical No. 8

Definition:

Write a program to explain mutable and immutable objects in Python.

Aim/Objective:

- **To understand the concept of mutable and immutable objects in Python.**
- **To identify mutable and immutable data types.**
- **To demonstrate modification of mutable objects.**
- **To understand why immutable objects cannot be changed directly.**
- **To differentiate between mutable and immutable objects with examples.**

Source Code:

Original List: [10, 20, 30]

Modified List: [100, 20, 30]

Enrolment/ Roll No. 92600584103

Original Tuple: (10, 20, 30)

Original String: vasu

Modified String: ratan

Output:

Original List: [10, 20, 30]

Modified List: [100, 20, 30]

Original Tuple: (10, 20, 30)

Original String: vasu

Modified String: ratan

Conclusion:

Thus, we learned the difference between mutable and immutable objects in Python.

We also understood how changes affect mutable and immutable data types.

Practical No. 9

Definition:**. Write a program to define and use user-defined functions with different types of arguments.**

Aim/Objective:

- **To understand the concept of user-defined functions in Python.**
- **To define and call functions according to requirements.**
- **To demonstrate different types of function arguments.**
- **To use positional, keyword, default, and variable-length arguments.**
- **To understand the importance of functions in making programs modular and reusable.**

Source Code:

Enrolment/ Roll No. 92600584103

```
def greet():
print("Hello, Welcome to Python!")
def add(a, b):
print("Addition:", a + b)
def student(name, course="MCA"):
print("Name:", name)
print("Course:", course)

def details(name, age):
print("Name:", name)
print("Age:", age)
def total(*numbers):
print("Total:", sum(numbers))
greet()
add(10, 20)
```

student("Samir")

student("vasu", "BCA")

details(age=22, name="jaynish")

total(10, 20, 30, 40)

Output:

Hello, Welcome to Python!

Addition: 30

Name: jaynish

Course: MCA

Name: vasu

Course: BCA

Name: ratan

Age: 22

Enrolment/ Roll No. 92600584103

Total: 100

Conclusion:

Thus, we learned how to define and use user-defined functions with different types of arguments.

We also understood how functions make Python programs simple, organized, and reusable.

Practical No. 10

Definition:

Write a program to demonstrate recursion using factorial or Fibonacci series.

Aim/Objectives:

● To understand the concept of recursion in Python.

● To define and use a recursive function.

● To demonstrate recursion using factorial or Fibonacci series.

● To understand the base condition and recursive call.

● To learn how recursive functions solve problems step by step.

Source Code:

```
def factorial(n):
if n == 0 or n == 1:
return 1
else:
return n * factorial(n - 1)
num = int(input("Enter a number: "))
result = factorial(num)
print("Factorial of", num, "is:", result)
```

Output:

```
Enter a number: 55
Factorial of 55 is:
12696403353658275925965100847566516959580321051449436762275840000000000000
```

Enrolment/ Roll No. 92600584103

Conclusion:

Thus, we learned how to implement recursion using factorial or Fibonacci series. We also

understood the importance of the base condition and recursive calls.